

* Church's Thesis & (Church-Turing Thesis)

- In 1936, two papers came up with definition of algorithms.
- Alonzo Church used lambda-calculus (λ -calculus) to define algorithms.
- It turns out that both definitions are equivalent.
- The connection between the informal concept of algorithms and the formal definitions is called the Church-Turing thesis.

Turing machine are relatively simple, they can have only finite state and they can read and write only one symbol at a time on the tape. But it turns out that Turing machine are extremely powerful. We have seen that, Turing machine can be built to add numbers, to multiply numbers and different computation. Although, it is difficult to compute a particular function using Turing machine than that of the algorithm.

Church thesis which states that given any problem that can be solved with an effective algorithm, there is a Turing machine that can solve this problem.

Nothing will be considered an algorithm if it cannot be rendered as TM. The principle that "TMs are formal version of algorithm and that no computational procedure will be considered an algorithm unless it can be represented as a TM" is known as Church's Thesis. It is universally accepted by computer scientists that Turing Machine is a mathematical model of an algorithm. It is just a hypothesis, not a theorem. So, we cannot prove it.

Recursive Language:

- A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The Turing machine will halt every time and give an answer (accepted or rejected) for each and every string input.

Example:

$L = \{a^n b^n c^n \mid n \geq 1\}$ is recursive because we can construct a Turing machine which will move to final state if the string is of the form $a^n b^n c^n$ else move to non-final state. So, TM will always halt in this case.

→ It is also known as Turing decidable language.

Recursively Enumerable Language

→ A language L is said to be a recursively enumerable language if there exists a Turing machine ~~TM~~ which will accept (and therefore halt) for all the input strings which are in L .

→ But may or may not halt for all input strings which are not in L .

→ It means TM can loop forever for the strings which are not a part of the language.

→ It is also known as Type-0 Language or Turing recognizable languages.

→ All regular, context-free and recursive languages are recursively enumerable.

Decidable Language

→ A language L is decidable if it is a recursive language. All decidable languages are recursive language and vice-versa.

Partially Decidable Language

A language L is partially decidable if L is a recursively enumerable language.

Undecidable Language:

- A language is undecidable if it is not decidable.
- An undecidable language may sometimes be partially decidable but not decidable.
- If a language is not even partially decidable, then there exists no Turing machine for that language.

Decidable problems:

A problem is decidable if we can construct a TM which will halt in finite amount of time for every input and give answer as 'yes' or 'no'. A decidable problem has an algorithm to determine the answer for a given input.

Examples:

1. Equivalence of two regular languages

Given any two regular languages, there is an algorithm and Turing machine to decide whether two regular languages are equal or not.

2. Finiteness of regular language:
Given a regular language, there is an algorithm and Turing machine to decide whether regular language is finite or not.

3. Emptiness of context-free language
Given a context-free language, there is an algorithm whether CFL is empty or not.

Undecidable problem:

A problem is undecidable if there is no Turing machine which will always halt in finite amount of time to give answer as 'yes' or 'no'.
An undecidable problem has no algorithm to determine the answer for a given input.

Examples:

1. Ambiguity of context-free languages:

Given a context-free language, there is no Turing machine which will always halt in finite amount of time and give answer whether language is ambiguous or not.

2. Equivalence of two context-free languages

Given two context-free languages, there is no Turing machine which will always halt in finite amount of time and give answer

whether two context-free languages are equal or not.

3. Everything or completeness of CFG:

Given a CFG and input alphabet, whether CFG will generate all possible strings of input alphabet (Σ^*) is undecidable.

* Reducibility and Undecidability

~~Language~~ or problem A is reducible to ~~language~~ or problem B if a solution to problem B can be used to solve a problem A.

For example, if A is the problem of finding some root of $x^4 - 3x^2 + 2 = 0$ and B is the problem of finding some root of $x^2 - 2 = 0$, then A is reducible to B. As $x^2 - 2$ is a factor of $x^4 - 3x^2 + 2$, a root of $x^2 - 2 = 0$ is also a root of $x^4 - 3x^2 + 2 = 0$.

Note: If A is reducible to B and B is decidable then A is decidable.

If A is reducible to B and A is undecidable then B is undecidable.

Halting Problem

Halting problem is the problem of finding or determining, from whether the program will finish, running or continue to run forever, from a description of an arbitrary computer program and an input.

For example,

in pseudocode, the program

while (true) continue
does not halt; rather it goes on forever
in an infinite loop. On the other hand,
the program
print "Hello, world!"
does halt.

* Given a Program, will it Halt?

* Given a Turing Machine, will it halt when run on some particular Turing machine given input string?

* Given some program written in some language (Java/C/etc) will it ever get into an infinite loop or will it ~~termi~~ always terminate?

Answers:

- In General we can't always know.
- The best we can do is run the program and see whether it halts.
- For many programs we can see that it will always halt or sometimes loop.

note: But For programs in general the question is undecidable.

Can we design a machine which if given a program can find out or decide if that program will always halt or not halt on a particular input?

⇒ Let us assume that we can?

$H(P, I)$

└ Halt

└ Not Halt

$H \rightarrow$ represents machine or program

$P \rightarrow$ program

$I \rightarrow$ Input

P and I are arguments.

→ This machine 'H' says whether 'P' will Halt or not halt.

→ This machine 'H' allows us to write another program 'C(x)'

$C(x)$

if $\{ H(x, x) == \text{Halt} \}$

Loop forever;

else

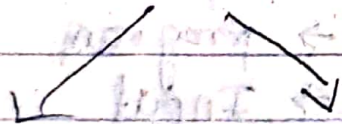
Return;

→ 'C' takes argument 'x' as a program.

→ In C, we called 'H' which takes two arguments as ~~$H(x, x)$~~ x & x where first 'x' is program and 2nd x as input.

Now, If we run 'C' on itself;

$C(C)$



$H(C, C) == \text{Halt}$

$H(C, C) == \text{Not Halt}$



Not Halt



Halt

→ If $H(C, C) == \text{Halt}$, we get loop for forever from $C(x)$, that means ~~Not~~ infinite. Which say Not Halt.

~~→ If $H(c, c)$~~

→ If $H(c, c) == \text{Not Halt}$, we get "return"
we not infinite, which says it halts.

→ This shows contradiction. so, Halting problem is
undecidable.

~~→ $H(c, c)$ Halting~~

Chapter 7 Computational Complexity Theory

Computable language

A computable language, then, is a formal language that can be accepted by some TM.

With respect to formal languages, the analogous concept to solvability is called decidability.

A language is decidable if some TM can take any string of the appropriate symbols and determine whether or not that string is a sentence of the language.

A language is acceptable, which we might also call computable, if some TM can determine if any string that is a sentence of the language is indeed a sentence, but for strings that are not sentence we cannot necessarily make any statement about that string at all. In this case, the membership problem for the language is partially solvable.

An un-acceptable language is non-computable language. That is, there is no any TM or no any algorithm, which can accept such languages.

* Computable function :

Computable functions are the basic objects of study in computability theory. A function is computable if there exists an algorithm that can do the job of function i.e. given an input of function domain it can return the corresponding ~~function~~ output.

According to the church-Turing thesis, computable functions are exactly the functions that can be calculated using a mechanical calculation device given unlimited amounts of time and storage space.

Equivalently, this ~~is~~ thesis states that a function is computable if and only if it has an algorithm. Note that an algorithm in this sense is understood to be a sequence of steps, a person with unlimited time and an unlimited supply of pen and paper could follow.

The class of computable functions can be defined in many equivalent models of computation, including

- Turing Machine
- recursive function
- Lambda calculus
- Register Machines

4 Complexity Theory :

Complexity theory considers not only whether a problem can be solved at all on a computer (as computable theory), but also how efficiently the problem can be solved. Thus, the complexity theory classifies problems according to their degree of difficulty. Two major aspects are considered: time complexity and space complexity, which are respectively how many steps does it take to perform a computation, and how much memory is required to perform that computation.

Complexity classes

A complexity class is the set of all of the computational problems which can be solved using a certain amount of a certain computational resource.

Class P and NP problems

* Class P (Polynomial time decidable problems)

P is a class of problems which can be decided in polynomial time i.e. there's an algorithm for such a problem which tells whether the solution of a given instance of a problem is true/false in $O(n^k)$ time.

← Tractable

for some constant K .

OR

A language L is said to be in class P if there exists a deterministic Turing machine M such that M is of time complexity $P(n)$ for some polynomial P and M accepts L .

P is the complexity class containing decision problems which can be solved by a deterministic Turing machine in polynomial time.

Decision problems are those whose answer can be given in Yes/No.

For example,

- finding the max element in array
- checking whether a string is palindrome or not
- checking a number is prime or not.
- finding shortest path using Kruskal's or Prim's algorithm
- finding minimum spanning tree and so on.

* Class NP (Non-deterministic polynomial time decidable Problems)

NP is a class of problems which can be solved by non-deterministic Turing machine in polynomial time. It is a set of problems that can be "verified" by a deterministic Turing machine in polynomial time.

OR

A language L is said to be in class NP if there exists a non-deterministic Turing machine M such that M is of time complexity $P(n)$ for some polynomial P and M accepts L .

The class of NP consists of all decision problems where 'yes' can be verified by a certifier.

A decision problem is in class NP, if we can solve it in polynomial time using a non-deterministic Turing machine to get the answer "yes" to your problem. That means we cannot solve the problem in polynomial time using a deterministic Turing machine. But we can always check whether our solution is right in polynomial time. So, if someone gives you an NP problem and the answer as "yes", we can check whether the answer is right or not in polynomial time. But keep in mind that we cannot find the answer in polynomial time (only check whether an answer is right).

for example,

Sudoku is an NP problem because if you can "quickly" verify a solution to a sudoku problem. (By "quickly"; I really mean "in polynomial time.") It doesn't matter if you can actually figure out the solution quickly or not. All that matters for a problem to be in NP is if a solution can be verified quickly.

NP-hard

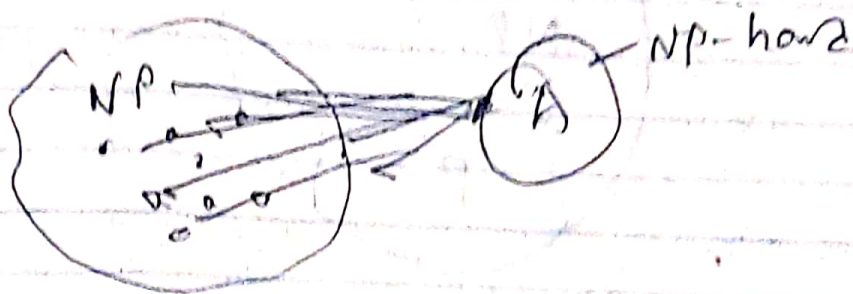
A problem is in NP-hard if every problem in NP can be "reduced" to that problem. (This seems to be opposite of "reduction", plus you'd have to take computational steps to reduce it - so it's requirement that the reduction itself is polynomial).

A lot of times you can solve a problem by reducing it to a different problem.

I can reduce problem A to problem B, if given a solution to problem B, I can easily construct a solution to problem A. (In this case, "easily" means "in polynomial time").

If a problem is NP-hard, this means I can reduce any problem in NP to that problem. This means if I can solve that problem I can easily solve any problem in NP. If we could solve an NP-hard problem in polynomial time, this would prove $P = NP$.

$P = NP$, which means any problem that can be verified in polynomial time can also be solved in polynomial time and vice-versa. If they could prove this, it would revolutionize computer science because people would be able to construct faster algorithms for a lot of important problems.

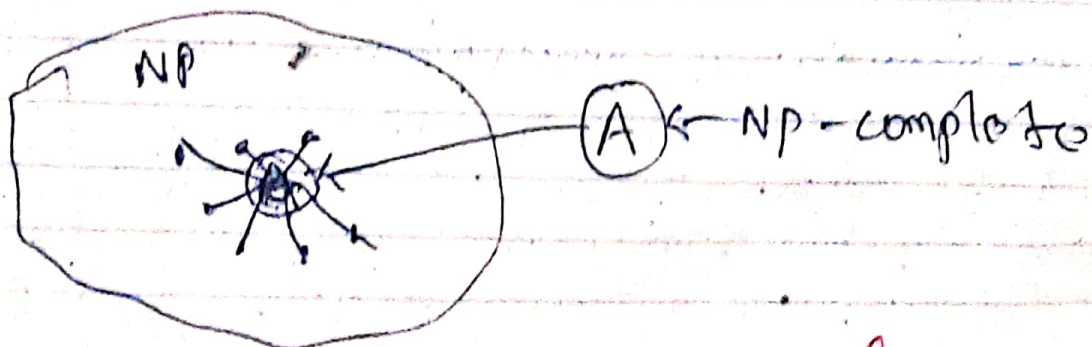


Example: vertex covering problem, Travelling Salesman.

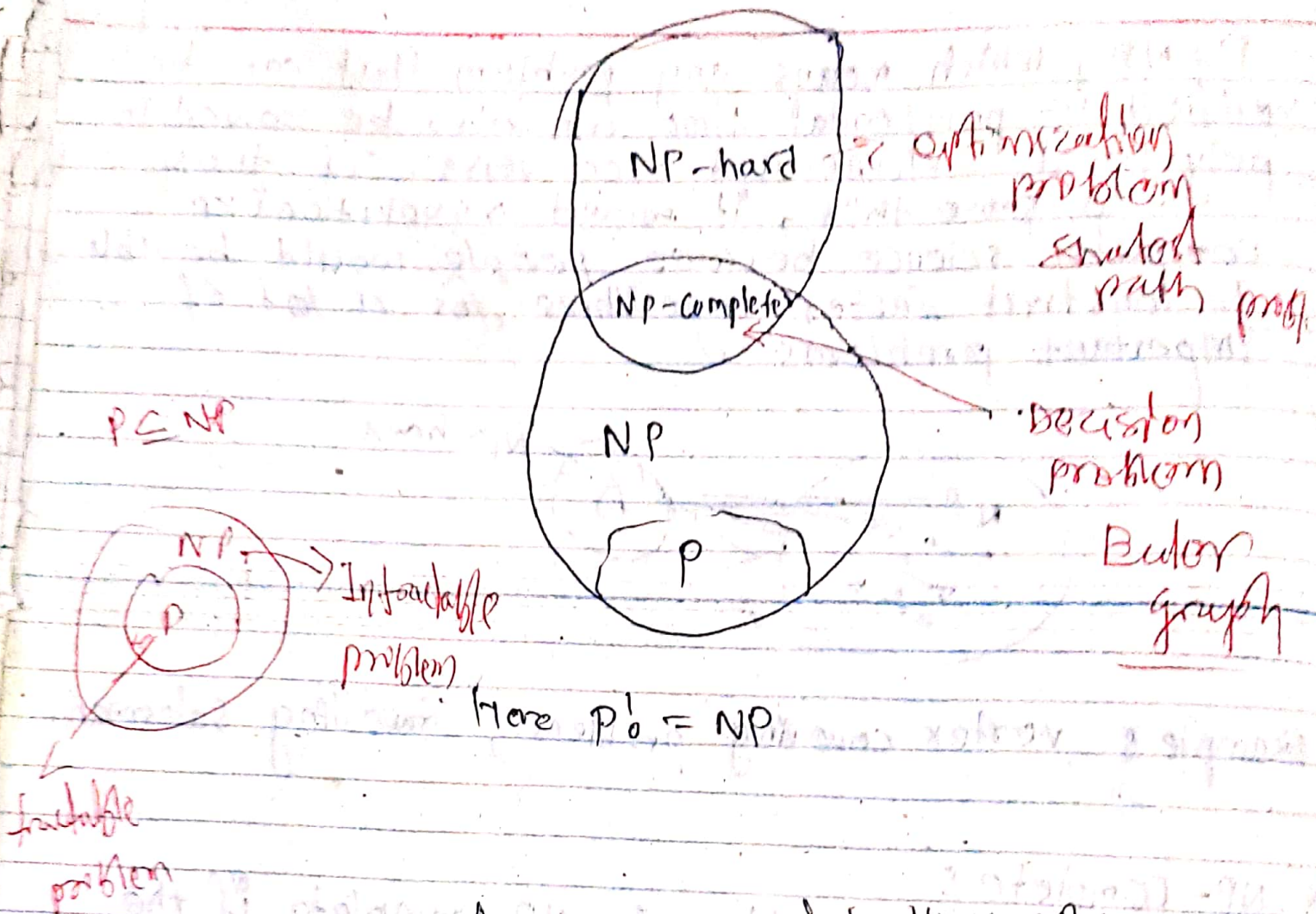
NP-complete:

A problem is NP-complete if the problem is both

- NP-hard, and
- In NP



Eg.: Whether a graph has a hamilton cycle,
Whether a boolean formula is satisfiable



All NP-hard cannot be NP-complete.

But all NP-complete are

NP-hard.